Why I Built FlowEngine: A Declarative Rules-Driven Wizard Engine for Ruby

Table of Contents

Ruby has an embarrassment of riches when it comes to state machines and workflow engines. What it **doesn't** have is a declarative, framework-agnostic engine for building **rules-driven, conditionally-branching questionnaires** — the kind of multi-step intake forms where the next question depends on the **answers** to previous ones. So I built one.

This post explains why none of the existing gems solved the problem I actually had, what FlowEngine does differently, and how you can use it today to build intelligent intake forms, qualification wizards, and branching surveys without coupling your flow logic to any framework or database.

The Problem

My wife is a tax professional. Her practice needs to qualify inbound leads — quickly determine the complexity of a prospective client's tax situation and route them to the right service tier. This is a **branching questionnaire**, not a linear form:

- If you have business income ask how many businesses
- If more than two businesses ask for EIN details
- If you have crypto ask about exchanges and transaction volume
- If charitable contributions exceed $5,000 ask for documentation

A simple W2 filer answers eight questions. A married filer with businesses, crypto, rentals, foreign accounts, and large charitable donations answers seventeen. **Same form, radically different paths through it.**

I looked at every gem I could find. None of them solved this.

The Landscape: What Already Exists

State Machine Gems: AASM, Statesman, Workflow

The most mature category. AASM is the 800-pound gorilla — actively maintained, 4,800+ stars, and battle-tested across thousands of Rails apps.

```
class Order
  include AASM

  aasm do
    state :pending, initial: true
    state :paid, :shipped, :cancelled

    event :pay do
      transitions from: :pending, to: :paid, guard: :payment_valid?
    end

    event :ship do
      transitions from: :paid, to: :shipped
    end
  end
end
```

AASM is **excellent** at what it does: modeling the lifecycle of a single object through well-defined states. Guards can conditionally allow or deny transitions.

Statesman from GoCardless adds database-backed transition history for auditability. The workflow gem offers a clean DSL with hooks and graphing support.

Why they don't solve the problem: These gems model **object lifecycle** — an order goes from `pending` to `paid` to `shipped`. They have no concept of **questions**, **answer collection**, or **data-dependent path selection across a graph of steps**. You could theoretically abuse a state machine into a questionnaire, but you'd be fighting the abstraction at every turn. The state lives on the **object**; in a questionnaire, the state is a **position in a directed graph** with accumulated context from previous answers driving the next transition.

Form Wizard Gems: Wicked, FormWizard, DVLA WizardFlow

Wicked is the go-to for multi-step forms in Rails. It's clean and well-integrated:

```
class AfterSignupController < ApplicationController
  include Wicked::Wizard
  steps :personal_info, :social_profiles, :confirmation

  def show
    @user = current_user
    render_wizard
  end

  def update
    @user = current_user
    @user.update(user_params)
    render_wizard @user
  end
end
```

FormWizard adds per-step validations and session persistence. The DVLA WizardFlow (from the UK Driver & Vehicle Licensing Agency) provides auto-generated routes from step definitions.

Why they don't solve the problem: All of these are **linear**. They define a fixed sequence of steps. Wicked lets you dynamically set steps at runtime, but there's no declarative rule DSL — no `contains()`, `all()`, `any()`, no composable boolean conditions evaluated against accumulated answers. You **can** shove conditional logic into your controller, but then you're writing imperative spaghetti instead of a declarative flow definition.

Workflow Engines: FlowCore, workflow_rb

FlowCore is the most architecturally ambitious entry — a Petri-net-based workflow engine for Rails with ArcGuards, TransitionTriggers, and STI-based extensibility. It's genuinely impressive engineering.

Why it doesn't solve the problem: FlowCore is an **orchestration** engine. It routes **tasks** through a system — think approval chains, document review pipelines, business process automation. It has no concept of a user-facing questionnaire, collected answers, or conditional branching based on those answers. It's also tightly coupled to Rails and ActiveRecord by design.

Rules Engines: Ruleby, Wongi Engine

Ruleby implements the Rete algorithm for forward-chaining production rules. Wongi Engine provides a DSL for rule-based reasoning over fact triplets.

Why they don't solve the problem: These are pure inference engines — they evaluate rules over facts. They don't model a user-facing flow, don't collect answers step by step, and don't manage a stateful walk through a graph. You **could** use one as a rule evaluator inside a flow engine (and FlowEngine's architecture actually allows for this), but on their own they're the wrong level of abstraction.

Survey Gems: Surveyor, Questionnaire

Surveyor is a Rails engine with its own survey DSL. The Survey gem stores questions in the database and manages attempts/scoring.

Why they don't solve the problem: Survey gems are designed for **data collection and scoring**, not for **conditional routing**. They store questions in a database and present them sequentially. Some support basic skip logic, but none offer a composable, AST-based rule system for arbitrarily complex branching conditions.

The Gap

Here's what I needed and couldn't find:

Capability	AASM	Wicked	FlowCore	Ruleby	FlowEngine
Declarative flow DSL					
Directed graph (not linear)				N/A	
Conditional branching on collected data	[1]		[2]		
AST-based composable rule system					
User-facing question/answer model					
Stateful answer collection & history		Partial			
Framework-agnostic (no Rails dependency)					
Immutable flow definitions		N/A		N/A	
Mermaid diagram export					
Pluggable validation adapters					

[1] AASM guards are method-based, not data-driven.

[2] FlowCore's ArcGuards are extensible but don't operate on accumulated user answers.

The gap is clear: no existing Ruby gem combines a declarative flow DSL, directed-graph traversal with conditional branching, an AST-based rule system, and stateful answer collection — all without framework dependencies.

What FlowEngine Does

FlowEngine fills that gap. Here's the thirty-second pitch:

> FlowEngine is a **declarative flow engine** for building **rules-driven wizards** and **intake forms** in pure Ruby. You define multi-step flows as **directed graphs** with **conditional branching**, evaluate transitions using an **AST-based rule system**, and collect structured answers through a **stateful runtime engine** — all without framework dependencies.

The Core Idea

A FlowEngine flow definition is a directed graph where:

- **Nodes** are steps (questions) with types, prompts, and options
- **Edges** are transitions with optional rule conditions
- **Rules** are immutable AST objects that evaluate against accumulated answers
- **The Engine** walks the graph, collecting answers and choosing the next step

```
definition = FlowEngine.define do
  start :name

  step :name do
    type :text
    question "What is your name?"
    transition to: :age
  end

  step :age do
    type :number
    question "How old are you?"
    transition to: :beverage, if_rule: greater_than(:age, 21)
    transition to: :thanks
  end

  step :beverage do
    type :single_select
    question "Pick a drink."
    options %w[Beer Wine Cocktail]
    transition to: :thanks
  end

  step :thanks do
```

```
    type :text
    question "Thank you!"
  end
end
```

If you're 25, you get the beverage question. If you're 18, you skip straight to thanks. The flow definition is **frozen and immutable** — no accidental mutations, safe to share across threads or serialize.

The Rule System

This is where FlowEngine diverges most sharply from everything else in the Ruby ecosystem. Rules are **AST objects** — not strings, not hashes, not procs. They're immutable, composable, and evaluate polymorphically:

```
# Atomic rules
contains(:income_types, "Business")     # Array inclusion
equals(:status, "married")              # Equality
greater_than(:count, 2)                 # Numeric comparison
less_than(:age, 18)                     # Numeric comparison
not_empty(:dependents)                  # Presence check

# Composite rules — nest arbitrarily
all(
  equals(:filing_status, "married_filing_jointly"),
  any(
    greater_than(:business_count, 3),
    contains(:income_types, "Rental")
  ),
  not_empty(:dependents)
)
```

Every rule implements `evaluate(answers)` where `answers` is a simple hash of `{ step_id ⇒ value }`. No magic, no method missing, no runtime metaprogramming. Just polymorphic dispatch on immutable data structures.

Why AST objects instead of procs or lambdas?

1. **Serializable** — you can dump a rule tree to JSON/YAML and reconstruct it
2. **Inspectable** — `rule.to_s` gives you a human-readable representation
3. **Composable** — `all()` and `any()` nest without limit
4. **Exportable** — the Mermaid exporter can label edges with rule descriptions

A Real-World Example: Tax Intake

Here's the flow I actually built for my wife's practice — a 17-step tax intake wizard:

```
tax_intake = FlowEngine.define do
  start :filing_status
```

```
  step :filing_status do
    type :single_select
    question "What is your filing status for 2025?"
    options %w[single married_filing_jointly
               married_filing_separately head_of_household]
    transition to: :dependents
  end

  step :income_types do
    type :multi_select
    question "Select all income types that apply."
    options %w[W2 1099 Business Investment Rental Retirement]
    transition to: :business_count,
               if_rule: contains(:income_types, "Business")
    transition to: :investment_details,
               if_rule: contains(:income_types, "Investment")
    transition to: :rental_details,
               if_rule: contains(:income_types, "Rental")
    transition to: :state_filing
  end

  step :business_count do
    type :number
    question "How many businesses do you own?"
    transition to: :complex_business_info,
               if_rule: greater_than(:business_count, 2)
    transition to: :business_details
  end

  # ... 14 more steps with conditional routing
end
```

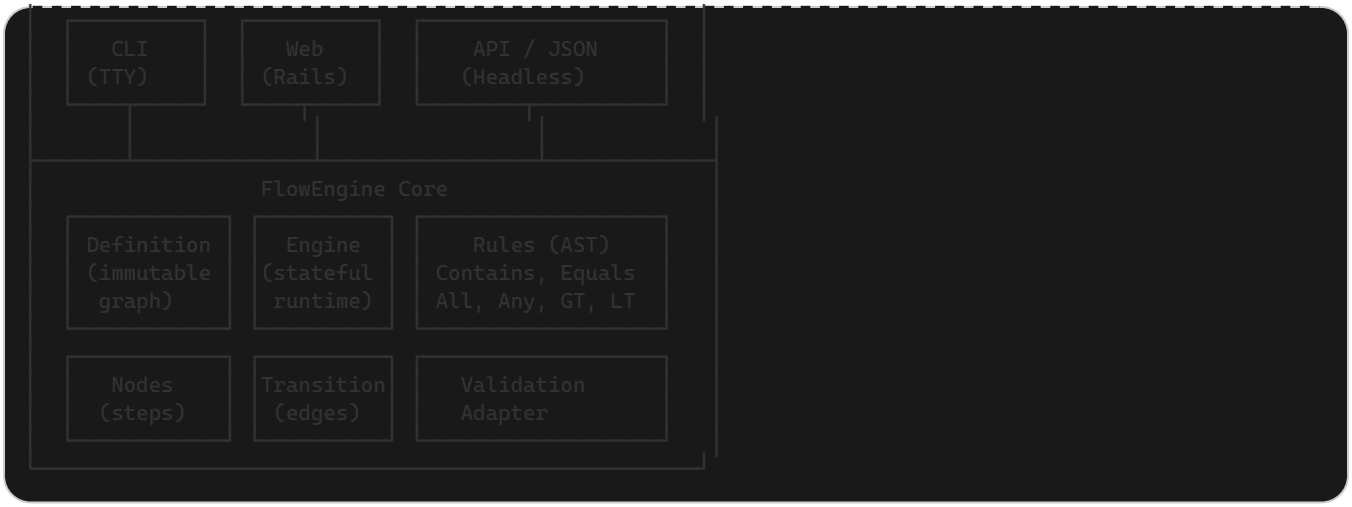
The path lengths vary dramatically:

Scenario	Steps	Skipped
Single filer, W2 only	8	9 steps skipped
Married, business + investments	12	5 steps skipped
Head of household, rentals + foreign accounts	10	7 steps skipped
Maximum complexity (all income types, high charitable)	17	Nothing skipped

Same form definition. Four completely different user experiences.

Architecture

FlowEngine's architecture follows one guiding principle: **separate flow logic from everything else**.



The core has **zero UI logic**, **zero DB logic**, and **zero framework dependencies**. It's pure Ruby, pure data structures, pure functions. Adapters handle rendering, persistence, and input collection:

Gem	Role
`flowengine` (core)	Pure Ruby engine — definitions, rules, evaluation, state
`flowengine-cli`	Terminal wizard adapter using TTY Toolkit + Dry::CLI
`flowengine-rails` **(coming soon)**	Rails Engine with ActiveRecord persistence and web views

The Ecosystem Vision

FlowEngine is the **engine**. What I'm building on top of it is the **car**:

Qualified.at — a service that lets non-technical users define branching intake forms through a visual editor, embed them on any website, and receive structured qualification data. Think Typeform, but with **real** conditional branching logic driven by an AST rule system, not just "skip to section 3."

The three-gem architecture makes this possible:

1. **`flowengine`** is MIT-licensed, framework-agnostic, and free forever
2. **`flowengine-cli`** is a developer tool for testing and debugging flow definitions locally
3. **`flowengine-rails`** powers the web application with persistence, session management, and embeddable views

Getting Started

Install the gems:

```
gem install flowengine flowengine-cli
```

Or add to your Gemfile:

```
gem "flowengine"
gem "flowengine-cli"  # optional: for terminal-based testing
```

Define a flow, run it, inspect the results:

```
require "flowengine"

definition = FlowEngine.define do
  start :name

  step :name do
    type :text
    question "What is your name?"
    transition to: :role
  end

  step :role do
    type :single_select
    question "What's your role?"
    options %w[Developer Designer Manager]
    transition to: :languages,
               if_rule: equals(:role, "Developer")
    transition to: :tools,
               if_rule: equals(:role, "Designer")
    transition to: :team_size
  end

  step :languages do
    type :multi_select
    question "Which languages do you use?"
    options %w[Ruby Python JavaScript Go Rust]
    transition to: :done
  end

  step :tools do
    type :multi_select
    question "Which tools do you use?"
    options %w[Figma Sketch Photoshop Illustrator]
    transition to: :done
  end

  step :team_size do
    type :number
    question "How large is your team?"
    transition to: :done
  end

  step :done do
```

```
    type :text
    question "Thanks! We'll be in touch."
  end
end

engine = FlowEngine::Engine.new(definition)
```

Export a Mermaid diagram to visualize your flow:

```
exporter = FlowEngine::Graph::MermaidExporter.new(definition)
puts exporter.export
```

Links

- FlowEngine on GitHub
- FlowEngine CLI on GitHub
- FlowEngine on RubyGems
- FlowEngine CLI on RubyGems

What's Next

I'm actively working on:

- `flowengine-rails` — Rails Engine with ActiveRecord models, Turbo Streams integration, and embeddable views
- **Visual flow editor** — a React-based DAG editor that exports FlowEngine DSL
- **LLM integration** — using language models to **generate** flow definitions from natural language descriptions of business processes
- **Qualified.at** — the hosted platform built on top of all of the above

If you're building intake forms, qualification wizards, onboarding flows, or any kind of conditionally-branching questionnaire in Ruby — give FlowEngine a look. It might save you from reinventing the wheel.

And if you have ideas, bug reports, or want to contribute — the issue tracker is open.

Konstantin Gredeskoul is a software engineer, 4x CTO, and the author of 30+ open-source Ruby gems with over 120M cumulative downloads. He lives in San Francisco and builds things at reinvent.one.